Mealy and Moore Machines

Camilo Andres Niño Amaya
Systems and Computer Engineering
UPTC
Sogamoso, Colombia
camilo.nino02@uptc.edu.co

***Abstract*—This paper presents a concise study of two classical finite-state machine models with output: the Mealy machine and the Moore machine. The report answers theoretical questions covering the conceptual description, formal definitions, and main applications of both models. The fourth question, corresponding to a Mealy-machine case study, is intentionally omitted according to the assignment instructions. The discussion emphasizes the central difference between both models: in a Mealy machine, the output depends on the present state and the present input, whereas in a Moore machine, the output depends only on the present state. The document concludes with a case study of a Moore machine, a comparative analysis, and practical construction details using JFLAP.**

***Index Terms*—Finite-state machine, Mealy machine, Moore machine, sequential circuit, finite-state transducer, JFLAP.**

I. Introduction

Finite-state machines (FSMs) are formal models for systems whose behavior can be represented by a finite set of internal states, input symbols, output symbols, and transition rules. When an FSM produces output symbols, it is commonly understood as a finite-state transducer rather than only as an acceptor of strings [1], [2]. Two classical output models are widely used in automata theory and digital design: the Mealy model, historically associated with Mealy's synthesis method for sequential circuits [3], and the Moore model, associated with Moore's work on sequential machines [4].

The distinction is relevant because both models can describe input-output behavior, but they associate outputs with different parts of the machine. In a Mealy machine, outputs are associated with transitions because the output function depends on the current state and input. In a Moore machine, outputs are associated with states because the output function depends only on the current state [5], [6].

II. Question 1: Description of a Mealy Machine

A Mealy machine is a finite-state machine with output in which the output produced at a given step is determined by two elements: the present internal state and the present input symbol. In digital-system notation, this idea is often written as

$$y(t) = f\big(S(t), X(t)\big), \tag{1}$$

where $S(t)$ is the present state, $X(t)$ is the present input, and $y(t)$ is the current output [5], [6]. Therefore, a Mealy machine behaves as a deterministic transducer that maps an input sequence into an output sequence.

In a state-transition diagram, a Mealy machine is usually represented by directed edges labeled with pairs of the form a/b, where a is the input symbol that enables the transition and b is the output symbol produced during that transition. This means that the output is naturally associated with the transition, not exclusively with the destination state.

III. Question 2: Formal Definition of a Mealy Machine

A deterministic Mealy machine can be formally defined as the 6-tuple

$$M = (Q, \Sigma, \Gamma, \delta, \lambda, q_0), \tag{2}$$

where:

- Q is a finite, nonempty set of states;
- Σ is a finite input alphabet;
- Γ is a finite output alphabet;
- $\delta : Q \times \Sigma \to Q$ is the state-transition function;
- $\lambda : Q \times \Sigma \to \Gamma$ is the output function;
- $q_0 \in Q$ is the initial state.

This formalization captures the defining property of the Mealy model: the output function has domain $Q \times \Sigma$, so an output symbol depends on both a state and the input being processed [1], [3].

IV. Question 3: Applications of Mealy Machines

Mealy machines are useful when the output of a system must depend on both the system condition and the input currently being received.

1) **Sequential digital controllers:** Used to synthesize sequential circuits whose control signals depend on input events and the current state [5], [7].
2) **Sequence detectors:** Convenient for detecting input patterns in serial streams, generating output on the exact transition where the final pattern symbol is read.
3) **Communication protocols:** Modeling logic where output actions (e.g., error flags) depend on both state and the most recent event.
4) **Lexical analysis:** Widely used as transducers in language processing to transform strings into output strings [2].

V. Question 5: Description of a Moore Machine

A Moore machine is a finite-state machine with output in which the output depends only on the present state. In digital-system notation, this is expressed as

$$y(t) = f\big(S(t)\big), \tag{3}$$

which contrasts with the Mealy equation [5], [6]. In a Moore-machine state diagram, each state is labeled with its associated output, and edges are labeled only by input conditions. When the machine enters a state, the output assigned to that state becomes the machine output. This makes Moore machines safer against transient input glitches in synchronous hardware.

VI. Question 6: Formal Definition of a Moore Machine

A deterministic Moore machine is defined as the 6-tuple

$$N = (Q, \Sigma, \Gamma, \delta, \lambda, q_0), \tag{4}$$

where:

- Q, Σ, Γ, δ, and q_0 are defined exactly as in the Mealy machine;
- $\lambda : Q \to \Gamma$ is the output function.

The essential formal difference from the Mealy definition is the domain of the output function: λ maps states directly to output symbols [1], [4].

TABLE I
Formal Difference Between Mealy and Moore Machines

Model	Output function	Output associated with
Mealy	$\lambda : Q \times \Sigma \to \Gamma$	Transition
Moore	$\lambda : Q \to \Gamma$	State

VII. Question 7: Applications of Moore Machines

Because Moore machines isolate their outputs from direct combinational paths linked to inputs, they are favored in systems requiring highly stable, synchronized signals [5]. Key applications include:

1) **Traffic Light Controllers:** The state of the traffic light purely dictates the output signals sent to the bulbs. Inputs (like pedestrian buttons or car sensors) only influence the next state transition, not the direct output, preventing dangerous mid-cycle flickering.
2) **Digital Counters and Clock Dividers:** The output of a digital counter is structurally identical to its current state.
3) **SRAM/Memory Controllers:** Hardware control logic where timing is critical, and outputs (like Write Enable or Read Enable) must be glitch-free and tied strictly to a stable clock edge.

VIII. Question 8: Case Study of a Moore Machine

Consider a **Binary Parity Checker** designed to output '1' if the number of '1's received in a serial bitstream is odd, and '0' if the number is even.

In a Moore machine implementation, the outputs are intrinsic to the states:

- **States (Q):** S_{even} (Even number of 1s), S_{odd} (Odd number of 1s).
- **Outputs (λ):** $\lambda(S_{even}) = 0$, $\lambda(S_{odd}) = 1$.
- **Transitions (δ):**
  - From S_{even}: Input '0' $\to S_{even}$; Input '1' $\to S_{odd}$.
  - From S_{odd}: Input '0' $\to S_{odd}$; Input '1' $\to S_{even}$.

In this design, the current parity is physically stored as the state. The output simply reflects which state the machine is currently occupying, completely independent of whatever the next incoming bit might be before the clock cycle completes.

IX. Question 9: Differences and Similarities

Both Mealy and Moore machines share several fundamental **similarities**:

- Both are finite-state transducers utilized to model sequential logic behavior.
- Both possess identical computational power. According to automata theory, any regular language or transduction computable by a Mealy machine can be computed by an equivalent Moore machine, and vice versa [1].
- Both mathematically require a finite set of states, a starting state, and input/output alphabets.

However, their structural distinctions create important **differences**:

- **Output Dependency:** Mealy outputs depend on $Q \times \Sigma$ (State and Input), whereas Moore outputs depend entirely on Q (State alone).
- **State Count:** A Mealy machine typically requires fewer states than an equivalent Moore machine, because a single state in a Mealy model can emit multiple different outputs depending on the transition [5].
- **Response Delay:** A Mealy machine can react to an input change instantly (asynchronously within the same clock cycle). A Moore machine inherently introduces a one-cycle delay, as the machine must first transition into a new state before the output updates.
- **Hardware Stability:** Moore machines are inherently immune to input combinational glitches, making them preferable when clean output signals are strictly required.

X. Question 10: Construction in JFLAP

JFLAP (Java Formal Languages and Automata Package) provides a graphical environment to construct and simulate finite-state transducers. To fulfill this requirement, two basic machines were designed. The following state transition tables provide the exact parameters used for their construction in the software.

A. Mealy Machine in JFLAP

The selected Mealy machine is a **Sequence Detector** that outputs a ‘1‘ whenever the sequence “11” is detected in a binary stream, and ‘0‘ otherwise. In JFLAP, when drawing a transition between states, the software prompts for the input and output in an `input;output` format on the edges.

TABLE II
TRANSITION TABLE FOR MEALY MACHINE (“11” SEQUENCE DETECTOR)

Present State	Next State		Output	
	Input = 0	Input = 1	Input = 0	Input = 1
q_0 (Initial)	q_0	q_1	0	0
q_1	q_0	q_1	0	1

The final implementation constructed in JFLAP is shown in Fig. 1.

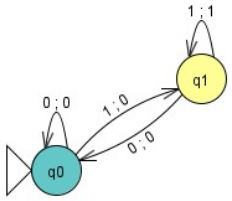

Fig. 1. Mealy Machine implementation in JFLAP.

B. Moore Machine in JFLAP

The selected Moore machine is the **Odd Parity Checker** described in the previous case study. In JFLAP, the output is assigned directly to the state, visually rendering the output in a box attached to the state circle. The edges only carry the input symbol.

TABLE III
TRANSITION TABLE FOR MOORE MACHINE (ODD PARITY CHECKER)

Present State	Next State		Output
	Input = 0	Input = 1	
q_0 (Even parity)	q_0	q_1	0
q_1 (Odd parity)	q_1	q_0	1

The final implementation constructed in JFLAP is shown in Fig. 2.

XI. CONCLUSION

Mealy and Moore machines are two foundational models of finite-state machines with output. A Mealy machine computes its output from the current state and the current input, so it is well suited for systems that must react immediately to input events and for minimizing state counts. A Moore machine computes its output only from the current state, so it is often easier to reason about in synchronous digital designs and protects against input-driven glitches. Their main formal

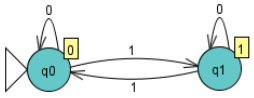

Fig. 2. Moore Machine implementation in JFLAP.

distinction is the mathematical domain of the output function. This theoretical difference practically dictates how outputs are represented in formal diagrams like JFLAP, how outputs are timed in hardware circuits, and how complex the control logic becomes.

REFERENCES

[1] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston, MA: Addison-Wesley, 2006.

[2] M. Mohri, “Finite-state transducers in language and speech processing,” *Computational Linguistics*, vol. 23, no. 2, pp. 269–311, 1997. [Online]. Available: https://aclanthology.org/J97-2003/

[3] G. H. Mealy, “A method for synthesizing sequential circuits,” *The Bell System Technical Journal*, vol. 34, no. 5, pp. 1045–1079, Sep. 1955.

[4] E. F. Moore, “Gedanken-experiments on sequential machines,” in *Automata Studies*, ser. Annals of Mathematics Studies, C. E. Shannon and J. McCarthy, Eds. Princeton, NJ: Princeton University Press, 1956, no. 34, pp. 129–153, reprinted by Princeton University Press/De Gruyter with DOI: 10.1515/9781400882618-006.

[5] P. P. Chu, *RTL Hardware Design Using VHDL: Coding for Efficiency, Portability, and Scalability*. Hoboken, NJ: John Wiley & Sons, 2006, chapter 10 discusses finite-state machine principles and practice.

[6] D. Mirza, “Lecture 14: Sequential networks – finite state machines: Moore and mealy,” CSE 140, University of California, San Diego, 2015. [Online]. Available: https://cseweb.ucsd.edu/classes/wi15/cse140-ab/slides/lec14_before.pdf

[7] Cornell University Department of Computer Science, “Finite state machines,” CS 3410 Computer System Organization and Programming, lecture slides, 2019. [Online]. Available: https://www.cs.cornell.edu/courses/cs3410/2019sp/schedule/slides/05-fsm-notes.pdf